

Small Errors, Big Consequences: Lessons from Data Mismanagement in Business

Karla Hernández 400916125

Fresenius University of Applied Science

Data Science and Data Analytics (WS 2025/26)

Prof. Dr. Stephan Huber

2025-02-17

Author Note

Correspondence concerning this article should be addressed to Karla Hernández
400916125, Email: hernandez.karla@stud.hs-fresenius.de

Abstract

Data plays a major role in business decisions, from production and hiring to marketing and forecasting. However, even tiny mistakes in a dataset can lead to completely wrong conclusions and very expensive consequences; so if the data has mistakes, the decisions will have mistakes. The challenge is that many data errors are small and easy to overlook, especially when working in RStudio, because R does not warn us when a result is logically wrong; it only warns us when the code fails. This handout explains how small issues such as missing values, duplicated rows, incorrect data types or poorly executed joins can quietly break calculations and models. Using simple examples of bad practice vs better practice in R, you'll learn how common habits can hide errors and how better habits prevent them. The goal is not about perfection, but about developing good routines that help us catch mistakes early, before they affect real business decisions.

Small Errors, Big Consequences: Lessons from Data Mismanagement in Business

Word count: 1264

1 “Small” Errors

When people think about data problems in business, they often imagine something dramatic; like a system failure, a cyberattack, or a huge coding bug. But in reality, business data disasters usually don’t come from big dramatic mistakes. They come from very small, very ordinary mistakes that slip through because nobody notices them.

Here’s why:

- Companies collect tons of data every day.
- Teams are under pressure to work fast and deliver results.
- Everyone assumes that software will catch mistakes automatically.
- R runs perfectly even if the data is wrong.

So a tiny mistake, like a missing value or a duplicated customer, might not look dangerous at first. But once that mistake is used in reports, dashboards, forecasts, or decisions... it suddenly becomes a big problem with big consequences.

2 What May Count as a “Small Error”?

Here are some examples of small mistakes that can break results later:

Table 1

What may be considered as “small error” examples

Error	What it Means
“Mexico” vs “México”	R treats them as two different categories
Number stored as text	Math doesn’t work the way we expect
Missing values	Averages and totals become wrong
Duplicate rows	Customers or employees counted twice
Wrong join	Rows multiply or disappear silently

3 Tiny Errors, Big Consequences

The previous mistakes look tiny, but later they affect totals, predictions, and business decisions. Let’s take now a look at how these errors can be related to real life examples:

Table 2*Examples in real life*

Area	Tiny Error	Big Result
Retail	Missing value in sales	Too little production that leads to lost on sales
HR	Employee duplicated during merge	Over-hired people that leads to unnecessary cost
Marketing	Duplicated customers	Overspent on ads

4 Bad Practice vs Better Practice in R

Now, we will see some examples on how to avoid small mistakes in R and replace risky habits with safer, clearer, and more reliable practices. Many of the “bad practices” shown below are considered wrong mainly because even though R runs them perfectly fine, the results are quietly broken. This means we will get broken results used in reports, dashboards, or business decisions, and here is when the damage is done.

This section is about learning to recognize moments where we *assume* something instead of **checking**, we let R “*guess*” instead of giving **clear instructions**, we work *faster* instead of **safer**, and we *trust* the output simply because “**there was no error message**”.

Now we’ll see some tasks and how we can avoid bad practices with examples of the best common practices:

1. **Importing:** R doesn’t convert text into categories automatically.

- *Bad Practice:* `read.csv("file.csv")`
- *Good Practice:* `read.csv("file.csv", stringsAsFactors = FALSE)`
- *Business Example:* Customer IDs don’t match, therefore sales linked to wrong clients.

2. **Missing values:** Get real averages instead of NA results that look “valid” but are unusable.

- *Bad Practice:* `mean(x)`
- *Good Practice:* `mean(x, na.rm = TRUE)`
- *Business Example:* Sales forecast too low that will lead to underproduction.

3. **Joining tables:** Avoid accidental duplicates and can see which rows didn't match.

- *Bad Practice:* `merge(df1, df2)`
- *Good Practice:* `left_join(df1, df2, by = "ID") + anti_join()`
- *Business Example:* HR turnover looks higher, by consequence there will be unnecessary hiring.

4. **Data types:** Verify and enforce data types instead of hoping R guessed correctly. *Only applicable for versions prior to R 4.0*

- *Bad Practice:* Assuming R knows the type
- *Good Practice:* `str()`, `glimpse()`, `mutate()`
- *Business Example:* Revenue calculations wrong leading to bad budgeting.

5. **Checking data:** Catch mistakes before modeling instead of discovering them too late.

- *Bad Practice:* Skipping inspection.
- *Good Practice:* `summary()`, `head()`, `table()`, `view()`
- *Business Example:* Management trusts a dashboard that is wrong.

6. **Filtering:** Keeps the dataset clean and consistent instead of returning unpredictable formats.

- *Bad Practice:* `data[data$age > 18]`
- *Good Practice:* `filter(data, age > 18)`
- *Business Example:* Customer segments misclassified.

7. **Removing duplicates:** Removes duplicates but also lets us see exactly which rows were duplicated.

- *Bad Practice:* `unique(data)`
- *Good Practice:* `distinct(data, .keep_all = TRUE)`
- *Business Example:* Marketing overspends on duplicated customers.

8. **Renaming columns:** Rename the correct column even if column order later changes.

- *Bad Practice:* `colnames(data) [3] <- "sales"`

- *Good Practice:* `rename(data, sales = old_name)`
 - *Business Example:* KPIs calculated using the wrong metric.
9. **Selecting columns:** Safer because column names don't change, column numbers do.
- *Bad Practice:* `data[, c(1,2,5)]`
 - *Good Practice:* `select(data, name, price, region)`
 - *Business Example:* Reports based on wrong variables.
10. **Dates:** Prevents R from guessing the wrong date format, which happens often.
- *Bad Practice:* `as.Date(Date)`
 - *Good Practice:* `as.Date(date, format="%d/%m/%Y")` or `lubridate::dmy()`
 - *Business Example:* Sales trends analyzed for wrong time period.
11. **Reading Excel:** Makes sure to always read the correct sheet even if the sheet order changes.
- *Bad Practice:* `read_excel("file.xlsx")`
 - *Good Practice:* `read_excel("file.xlsx", sheet="Sales2024")`
 - *Business Example:* Decisions based on outdated data.
12. **Plotting:** Control exactly what gets plotted instead of letting R “guess”.
- *Bad Practice:* `plot(data)`
 - *Good Practice:* `ggplot(data, aes(x, y)) + geom_line()`
 - *Business Example:* Executives draw wrong conclusions from graphs.
13. **Pipelines (chaining steps):** A pipeline shows the full logic in one place, makes the order of steps clear, and makes the code easier to read, review, and debug.
- *Bad Practice:* Many separate transformations that overwrite the dataset multiple times.
 - *Good Practice:* Use a single pipeline to apply all steps in order.
 - *Business Example:* Losing track of transformations.
14. **Saving:** Always keep the original data safe in case of errors.
- *Bad Practice:* Overwriting the raw file

- *Good Practice:* Save a separate clean file
- *Business Example:* Errors cannot be traced or corrected later.

By comparing bad practices vs better practices, we can clearly see how very small choices in code, tiny details, can be the difference between a *result that is silently wrong* and a **result that can be trusted**.

5 Lessons from Data Mismanagement in Business

1. A tiny mistake in data can cause a huge mistake in a business decision.
2. R won't warn you when the result is wrong, only when the code is.
3. Never assume the data is clean, always check it.
4. Good analysis starts before modeling with data validation.
5. Being explicit in your code prevents disasters.
6. Documentation and reproducible scripts protect you from mistakes.
7. Joins are one of the most dangerous operations, check them twice.
8. Data types matter more than most people realize.
9. Peer review is essential.
10. Great analysts aren't those who make no mistakes, they are the ones who catch mistakes early.

6 Exercise

6.1 Scenario

You work in the analytics team of a retail company and your manager asks the following: “*What is the average purchase amount by age group?*” Using the file `shopping_behavior.csv`, calculate:

1. The average purchase for customers under 25
2. The average purchase for customers 25–40
3. The average purchase for customers over 40

Click to download dataset: CSV shopping_behavior dataset

6.2 Solution

```
# 1) Load packages
library(dplyr)
library(ggplot2)

# 2) Import data
data_raw <- read.csv(
  "shopping_behavior.csv",
  stringsAsFactors = FALSE,
  check.names = FALSE
)

# 3) Remove duplicates
data_unique <- data_raw %>%
  distinct(.keep_all = TRUE)

# 4) Create age groups
data_age <- data_unique %>%
  mutate(
    age_group = case_when(
      Age < 25 ~ "Under 25",
      Age >= 25 & Age <= 40 ~ "25-40",
      Age > 40 ~ "Over 40",
      TRUE ~ "Unknown"
    )
  )

# 5) Calculate average purchase
summary_table <- data_age %>%
  group_by(age_group) %>%
  summarise(
    avg_purchase = mean(`Purchase Amount (USD)`, na.rm = TRUE),
    n_customers = n(),
```



```
.groups = "drop"
) %>%
  arrange(age_group)

# 6) Round values
summary_table$avg_purchase <- round(summary_table$avg_purchase, 2)

# 7) Display results
print(summary_table)

# 8) Simple plot
ggplot(summary_table, aes(x = age_group, y = avg_purchase)) +
  geom_bar(stat = "identity") +
  labs(
    title = "Average Purchase Amount by Age Group",
    x = "Age Group",
    y = "Average Spend (USD)"
  ) +
  theme_minimal()
```

6.3 Outcome

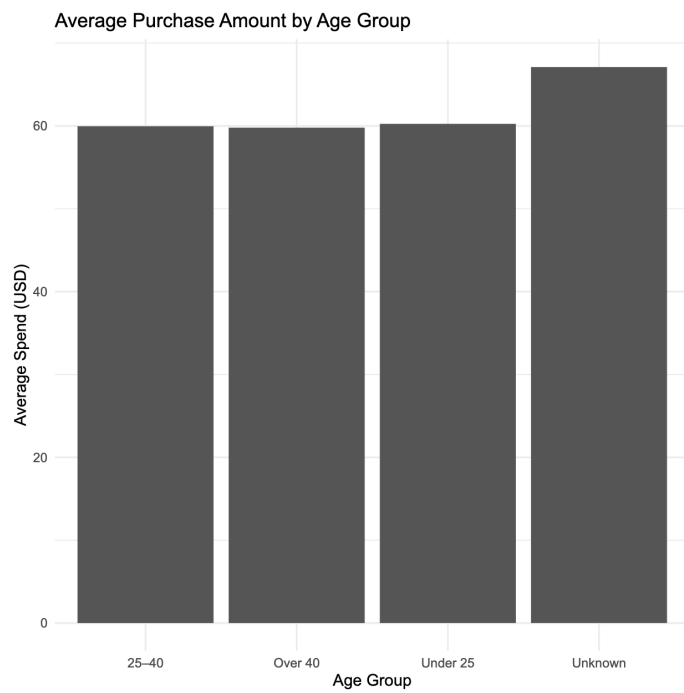
Figure 1

Average purchase per group

	age_group	avg_purchase	n_customers
	<chr>	<dbl>	<int>
1	25–40	60.0	1187
2	Over 40	59.8	2218
3	Under 25	60.2	485
4	Unknown	67.1	10

Figure 2

Simple bar plot



7 References

References marked with an asterisk indicate studies included in the meta-analysis.

Jung, D. (2024). Business data understanding. In *The modern business data analyst: A case study introduction into business data analytics with CRISP-DM and r* (pp. 49–110). Springer.

Masked Citation. (n.d.). *Masked Title*.

Wickham, H., & Grolemund, G. (2023). *R for data science* (2e). <https://r4ds.hadley.nz/>

Appendix

Affidavit

I hereby affirm that this submitted paper was authored unaided and solely by me. Additionally, no other sources than those in the reference list were used. Parts of this paper, including tables and figures, that have been taken either verbatim or analogously from other works have in each case been properly cited with regard to their origin and authorship. This paper either in parts or in its entirety, be it in the same or similar form, has not been submitted to any other examination board and has not been published.

I acknowledge that the university may use plagiarism detection software to check my thesis. I agree to cooperate with any investigation of suspected plagiarism and to provide any additional information or evidence requested by the university.

Checklist:

- ☒ The handout contains 3-5 pages of text.
- ☒ The submission contains the Quarto file of the handout.
- ☒ The submission contains the Quarto file of the presentation.
- ☒ The submission contains the HTML file of the handout.
- ☒ The submission contains the HTML file of the presentation.
- ☒ The submission contains the PDF file of the handout.
- ☒ The submission contains the PDF file of the presentation.
- ☒ The title page of the presentation and the handout contain personal details (name, email, matriculation number).
- ☒ The handout contains a bibliography, created using BibTeX with an APA citation style.
- ☒ Either the handout or the presentation contains R code that proofs the expertise in coding.
- ☒ The filled out Affidavit.
- ☒ The link to the presentation and the handout published on GitHub.
- ☐ In group work, each student's contribution is clearly defined, and individual performance can be assessed using specified sections, page numbers, or other objective criteria.

[Karla Hernández,] [2025-02-17,] [Köln, Germany]